

W4111 – Introduction to Databases
Section 002, Fall 2025
Lecture 9: Module II
DBMS Architecture (3), NoSQL (3)



Introduction

Contents

Content

Course Overview and Discussion

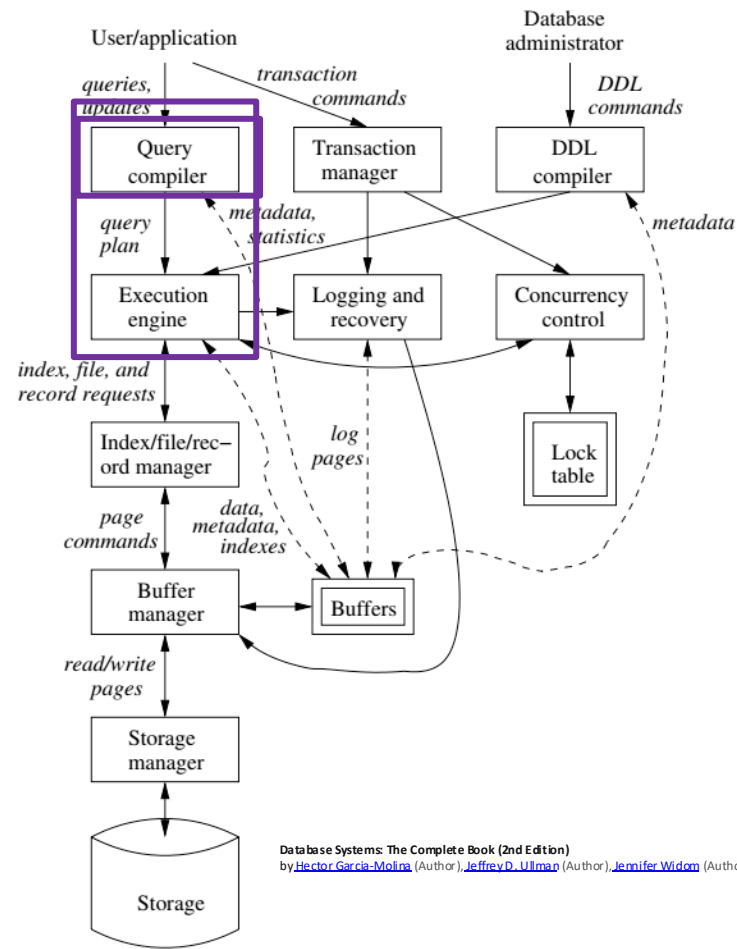
Course Overview and Discussion

Module II

Introduction and Reminder

Today

- Query Processing
- Query Optimization



Query Compilation

Query Compilation

Preview of Query Compilation

Database Systems: The Complete Book (2nd Edition) 2nd Edition

by [Hector Garcia-Molina](#) (Author), [Jeffrey D. Ullman](#) (Author), [Jennifer Widom](#) (Author)

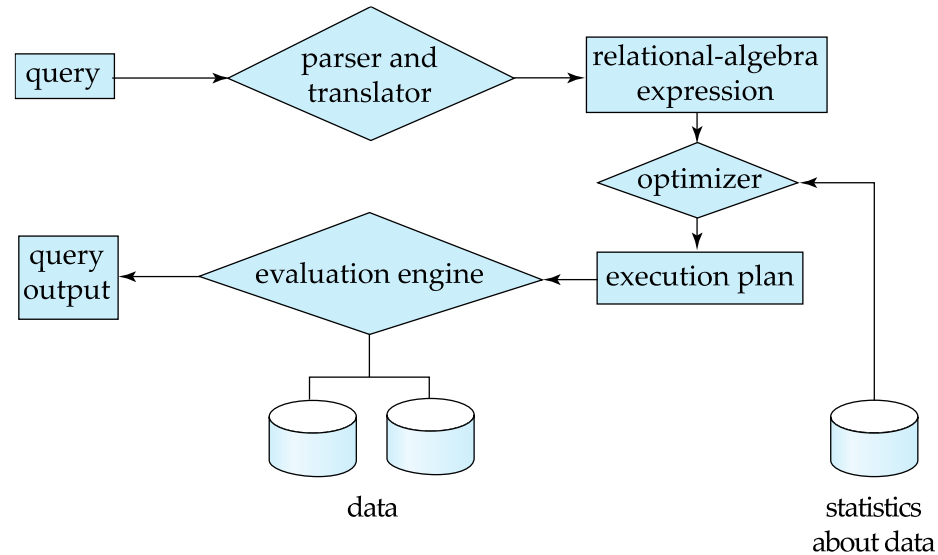
To set the context for query execution, we offer a very brief outline of the content of the next chapter. Query compilation is divided into the three major steps shown in Fig. 15.2.

- a) *Parsing*. A *parse tree* for the query is constructed.
- b) *Query Rewrite*. The parse tree is converted to an initial query plan, which is usually an algebraic representation of the query. This initial plan is then transformed into an equivalent plan that is expected to require less time to execute.
- c) *Physical Plan Generation*. The abstract query plan from (b), often called a *logical query plan*, is turned into a *physical query plan* by selecting algorithms to implement each of the operators of the logical plan, and by selecting an order of execution for these operators. The physical plan, like the result of parsing and the logical plan, is represented by an expression tree. The physical plan also includes details such as how the queried relations are accessed, and when and if a relation should be sorted.

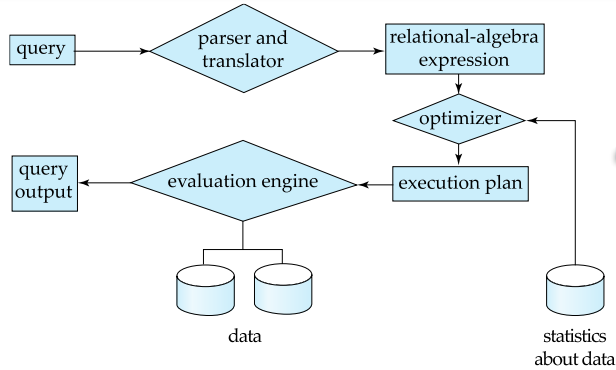


Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



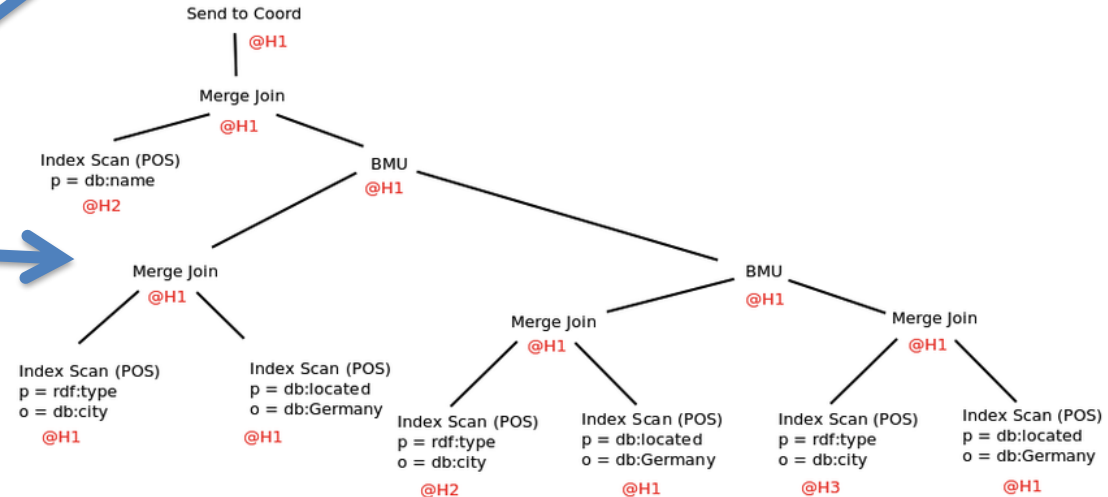
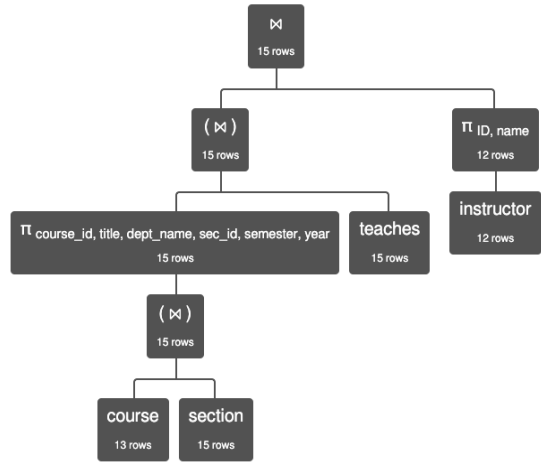
Query Processing



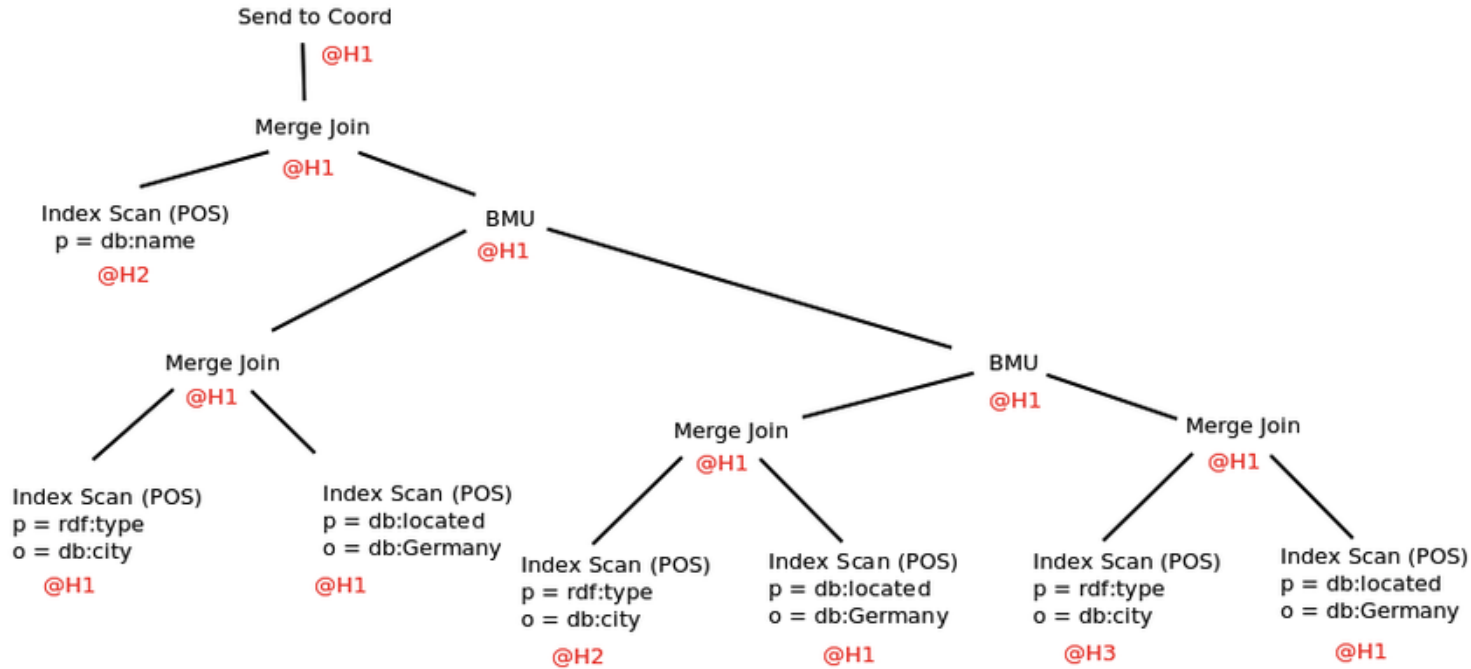
```

1 (
2   (π course_id, title, dept_name, sec_id, semester, year
3     (course ⋈ section)
4   )
5   ⋈ teaches
6 )
7 ⋈
8 (π ID, name (instructor))

```



Query Plan and Algorithms



Parsing and Execution

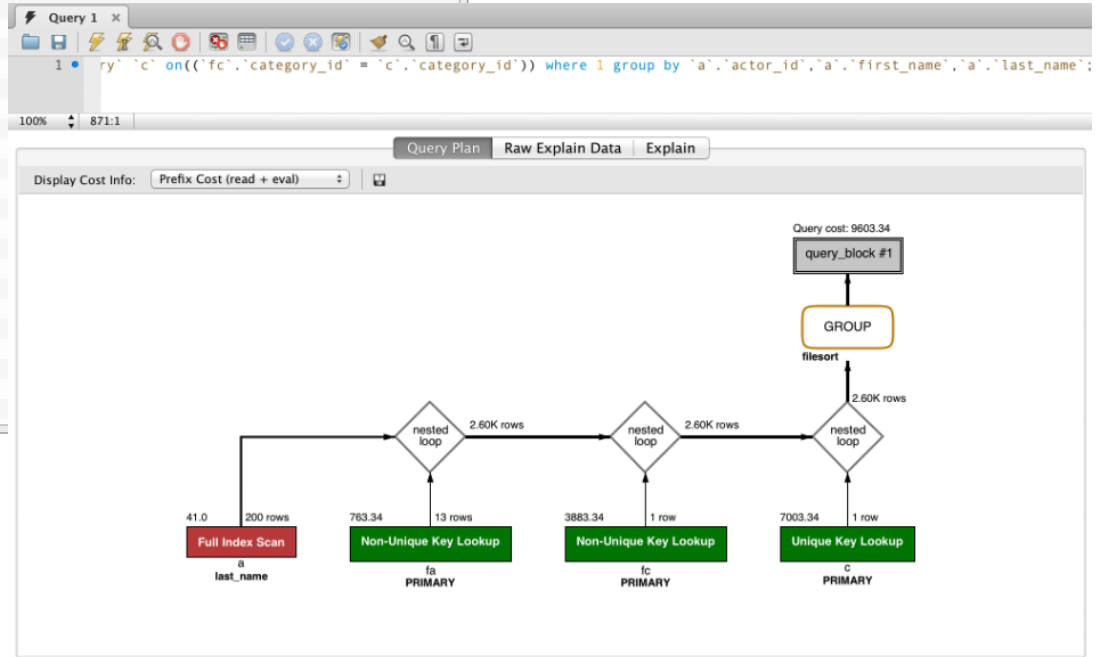
- Parser/Translator
 - Verifies syntax correctness and generates a *parse tree*.
 - Converts to *logical plan tree* that defines how to execute the query.
 - Tree nodes are *operator(tables, parameters)*
 - Edges are the flow of data “up the tree” from node to node.
- Optimizer
 - Modifies the logical plan to define an improved execution.
 - Query rewrite/transformation.
 - Determines *how* to choose among multiple implementations of operators.
- Engine
 - Executes the plan
 - May modify the plan to *optimize* execution, e.g. using indexes.

EXPLAIN Example

JSON EXPLAIN

Example JSON Formatted EXPLAIN

```
mysql> explain format=json select `a`.`actor_id` AS `actor_id`,`a`.`first_name` AS `first_name`,`a`.`last_name` AS `last_name`
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "9603.34"
    },
    "grouping_operation": {
      "using_filesort": true,
      "cost_info": {
        "sort_cost": "2600.00"
      },
      "nested_loop": [
        {
          "table": {
            "table_name": "a",
            "access_type": "index",
            "possible_keys": [
              "last_name",
              "ln_fn_idx"
            ],
            "key": "last_name",
            "used_key_parts": [
              "last_name",
              "first_name"
            ]
          }
        }
      ]
    }
  }
}
```





Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
 - Use an index on *salary* to find instructors with salary < 75000,
 - Or perform complete relation scan and discard instructors with salary ≥ 75000



Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Cost is estimated using statistical information from the database catalog
 - e.g.. number of tuples in each relation, size of tuples, etc.
- In this chapter we study
 - How to measure query costs
 - Algorithms for evaluating relational algebra operations
 - How to combine algorithms for individual operations in order to evaluate a complete expression
- In Chapter 16
 - We study how to optimize queries, that is, how to find an evaluation plan with lowest estimated cost

Algorithm Selection for SELECT



Selection Operation

This seems to assume that all blocks for the relation are on the same cylinder.

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$
- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition

- $Cost = (h_i + 1) * (t_T + t_S)$

- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.

- Records will be on consecutive blocks
 - Let b = number of blocks containing matching records

- $Cost = h_i * (t_T + t_S) + t_S + t_T * b$

- Replace $(t_T + t_S)$ with t_B , the time/cost for reading a block.
- In my way of thinking, h_i is the “height” of the index tree.
- The cost to find the index block is $h_i * t_B$. We then read the referenced data block, which happens once because it is a key.
- If it is a non-key, we may retrieve multiple records → reading multiple blocks.
- But, clustering index on the non-key field → the data records are sorted by the non-key → records are probably on consecutive sectors on the same track → one seek and then some number of block reads.





Selections Using Indices

- **A4 (secondary index, equality on key/non-key).**
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!

- $h_i * t_B$ to get the leaf node in the index.
- Each entry in the leaf node may reference a different block in the data file.
- If there are n matching records, we get
- $(h_i + n) * t_B$



Selections Involving Comparisons

- Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
 - a linear file scan,
 - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
 - For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index
- **A6 (clustering index, comparison).**
 - For $\sigma_{A \geq V}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
 - For $\sigma_{A \leq V}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
 - In either case, retrieve records that are pointed to
 - requires an I/O per record; Linear file scan may be cheaper!



Implementation of Complex Selections

- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- **A7 (conjunctive selection using one index).**
 - Select a combination of θ_i and algorithms A1 through A7 that results in the least cost for $\sigma_{\theta_i}(r)$.
 - Test other conditions on tuple after fetching it into memory buffer.
- **A8 (conjunctive selection using composite index).**
 - Use appropriate composite (multiple-key) index if available.
- **A9 (conjunctive selection by intersection of identifiers).**
 - Requires indices with record pointers.
 - Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.
 - Then fetch records from file
 - If some conditions do not have appropriate indices, apply test in memory.



Algorithms for Complex Selections

- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.
- **A10 (disjunctive selection by union of identifiers).**
 - Applicable if *all* conditions have available indices.
 - Otherwise use linear scan.
 - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
 - Then fetch records from file
- **Negation:** $\sigma_{\neg\theta}(r)$
 - Use linear scan on file
 - If very few records satisfy $\neg\theta$, and an index is applicable to θ
 - Find satisfying records using index and fetch from file

Conjunctive and Disjunctive



- Assume there is an index on *name* but not on *dept_name*.
- Consider two simple use cases
 - *select * from student where name="Smith" AND dept_name="Comp. Sci."*
 - *select * from student where name="Smith" OR dept_name="Comp. Sci."*
- With a conjunction, the engine must
 - Find and read the blocks with the correct name.
 - Scan each block to evaluate the rest of the AND, but this is not an I/O.
- The index does not help for the disjunction because the engine still needs to scan the data file and read blocks looking for the matching OR conditions.

Algorithm Selection for JOIN



Join Operation

- Several different algorithms to implement joins
 - Nested-loop join
 - Block nested-loop join
 - Indexed nested-loop join
 - Merge-join
 - Hash-join
- Choice based on cost estimate
- Examples use the following information
 - Number of records of *student*: 5,000 *takes*: 10,000
 - Number of blocks of *student*: 100 *takes*: 400



Nested-Loop Join

- To compute the theta join $r \bowtie_{\theta} s$
 for each tuple t_r **in** r **do begin**
 for each tuple t_s **in** s **do begin**
 test pair (t_r, t_s) to see if they satisfy the join condition θ
 if they do, add $t_r \cdot t_s$ to the result.
 end
 end
- r is called the **outer relation** and s the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.

For clarity (or confusion), I sometimes use the terms:

- Scan table for the outer relation
- Probe table for the inner relation



Nested-Loop Join (Cont.)

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is

$n_r * b_s + b_r$ block transfers, plus $n_r + b_r$ seeks

- If the smaller relation fits entirely in memory, use that as the inner relation.

- Reduces cost to $b_r + b_s$ block transfers and 2 seeks

- Assuming worst case memory availability cost estimate is

- with *student* as outer relation:

- $5000 * 400 + 100 = 2,000,100$ block transfers,

- $5000 + 100 = 5100$ seeks

- with *takes* as the outer relation

- $10000 * 100 + 400 = 1,000,400$ block transfers and 10,400 seeks

- If smaller relation (*student*) fits entirely in memory, the cost estimate will be 500 block transfers.
- Block nested-loops algorithm (next slide) is preferable.

R JOIN S on (...)

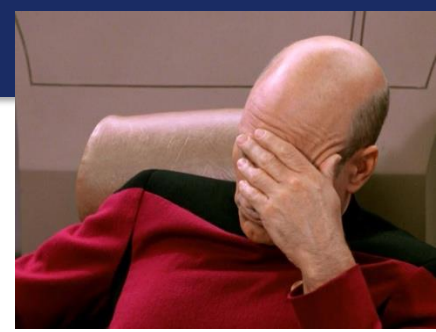
S JOIN R on (...)

Well, That was a Clear as Mud

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is

$n_r * b_s + b_r$ block transfers, plus $n_r + b_r$ seeks

- SELECT * from L JOIN R on L.x=R.y
- The algorithm reads each block of L one time and never reads it again in JOIN.
- For each record l in L, the algorithm must read each block of R one time.
- The number of blocks in a relation S is, over simplistically,
 - #(*S*) is the number of records.
 - *s*(*S*) is the average record size.
 - *S*(*b*) is the block size ➔ number of blocks is #(*S*)/(*s*(*b*)/*s*(*S*)), which is $O(\#(S))$
 - So, the number of blocks read is $O(\#(L)) * O(\#(R)) = O(\#(L) * \#(R))$
- “shmath” – Something that is completely made up but presented as precise and accurate using math symbols.





Indexed Nested-Loop Join

- Index lookups can replace file scans if
 - join is an equi-join or natural join and
 - an index is available on the inner relation's join attribute
 - Can construct an index just to compute a join.
- For each tuple t_r in the outer relation r , use the index to look up tuples in s that satisfy the join condition with tuple t_r .
- Worst case: buffer has space for only one page of r , and, for each tuple in r , we perform an index lookup on s .
- Cost of the join: $b_r (t_r + t_s) + n_r * c$
 - Where c is the cost of traversing index and fetching all matching s tuples for one tuple of r
 - c can be estimated as cost of a single selection on s using the join condition.
- If indices are available on join attributes of both r and s , use the relation with fewer tuples as the outer relation.

$$r.X = s.Y \text{ AND } r.P = s.Q$$
$$(X, Y) \rightarrow (P, Q)$$



Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.
 - For relations that fit in memory, techniques like quicksort can be used.
 - For relations that don't fit in memory, **external sort-merge** is a good choice.
-
- The engine may build either a sorted index or a hash index.
 - The key idea is:
 - The cost of building the index and then using the index in the algorithm
 - Is cheaper than running the default algorithm with an index

We Really Hate this Class

- One way to *overly* simplify these analyses is realizing that the no. of blocks is “linear” with the number of records/tuples in a relation. →
 - Overly simplify and assume that the block size is one record.
 - Analyze using the number of reads as a function of no. of rows.
- Assume that relation R has $\#(R)$ rows:
 - A tree-based lookup when you can use a key is $O(\log(\#R))$.
The base of the logarithm does not materially change the analysis.
 - A hash-based lookup is $O(1)$.
 - A scan or scan-based look up is $O(\#R)$.
- Index creation cost:
 - Hash is $O(\#(R))$
 - Tree is $O(\#(R) * \log(\#R))$
- So, very roughly an index join is either
 - $O(\#(L) * \log(\#(R)) + O(\#(R) * \log(\#(R))) = \log(\#(R)) * (\#(L) + \#(R))$ or
 - $O(\#(L) + \#(R))$, but this only works for equality.



$\#(R) = 100$

$\#(S) = 1000000$

Build index = 6 million

700 + 6 million



Merge-Join

1. Sort both relations on their join attribute (if not already sorted on the join attributes).
2. Merge the sorted relations to join them
 1. Join step is similar to the merge stage of the sort-merge algorithm.
 2. Main difference is handling of duplicate values in join attribute — every pair with same value on join attribute must be matched
 3. Detailed algorithm in book

	$a1$	$a2$		$a1$	$a3$		
pr	→	a	3	ps	→	a	A
		b	1			b	G
		d	8			c	L
		d	13			d	N
		f	7			m	B
		m	5				
		q	6				
		r				s	

$\#(A) = n, \#(B) = m$

$n * m$

$n * \log(n) + m * \log(m) + n + m$



Merge-Join (Cont.)

- Can be used only for equi-joins and natural joins
- Each block needs to be read only once (assuming all tuples for any given value of the join attributes fit in memory)
- Thus the cost of merge join is:
$$b_r + b_s \text{ block transfers} + \lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil \text{ seeks}$$

+ the cost of sorting if relations are unsorted.
- **hybrid merge-join:** If one relation is sorted, and the other has a secondary B⁺-tree index on the join attribute
 - Merge the sorted relation with the leaf entries of the B⁺-tree .
 - Sort the result on the addresses of the unsorted relation's tuples
 - Scan the unsorted relation in physical address order and merge with previous result, to replace addresses by the actual tuples
 - Sequential scan more efficient than random lookup



Hash-Join

- Applicable for equi-joins and natural joins.
- A hash function h is used to partition tuples of both relations
- h maps *JoinAttrs* values to $\{0, 1, \dots, n\}$, where *JoinAttrs* denotes the common attributes of r and s used in the natural join.
 - $r_0, r_1, \dots, r_n$ denote partitions of r tuples
 - Each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[\text{JoinAttrs}])$.
 - $r_0, r_1, \dots, r_n$ denotes partitions of s tuples
 - Each tuple $t_s \in s$ is put in partition s_i , where $i = h(t_s[\text{JoinAttrs}])$.
- *Note:* In book, Figure 12.10 r_i is denoted as H_{ri} , s_i is denoted as H_{si} and n is denoted as n_h .

On $A.x=b.y$ and $A.z=b.q$

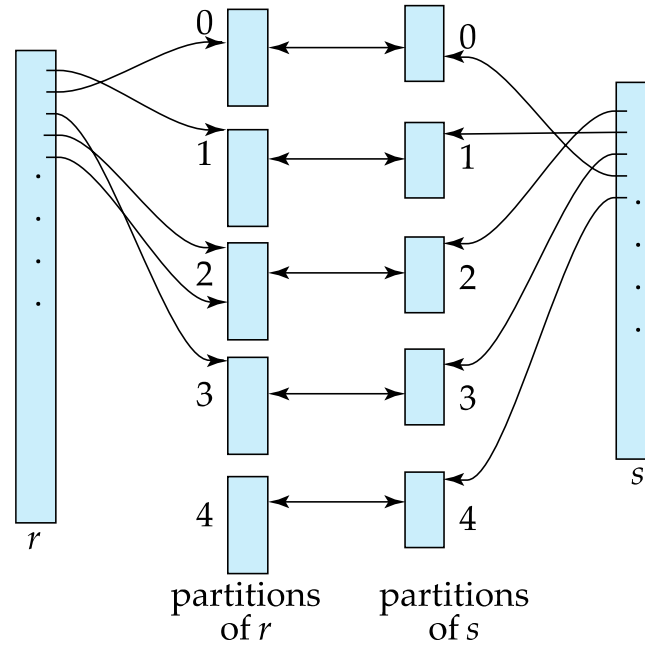
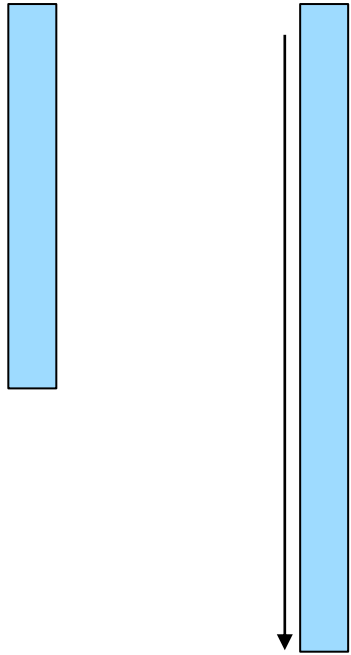
(z,q)

(x,y)



Hash-Join (Cont.)

On $a.X=b.Z$





Hash-Join Algorithm

The hash-join of r and s is computed as follows.

1. Partition the relation s using hashing function h . When partitioning a relation, one block of memory is reserved as the output buffer for each partition.
2. Partition r similarly.
3. For each i :
 - (a) Load s_i into memory and build an in-memory hash index on it using the join attribute. This hash index uses a different hash function than the earlier one h .
 - (b) Read the tuples in r_i from the disk one by one. For each tuple t_r locate each matching tuple t_s in s_i using the in-memory hash index. Output the concatenation of their attributes.

Relation s is called the **build input** and r is called the **probe input**.



Hash-Join algorithm (Cont.)

- The value n and the hash function h is chosen such that each s_i should fit in memory.
 - Typically n is chosen as $\lceil b_s/M \rceil * f$ where f is a “**fudge factor**”, typically around 1.2
 - The probe relation partitions s_i need not fit in memory
- **Recursive partitioning** required if number of partitions n is greater than number of pages M of memory.
 - instead of partitioning n ways, use $M - 1$ partitions for s
 - Further partition the $M - 1$ partitions using a different hash function
 - Use same partitioning method on r
 - Rarely required: e.g., with block size of 4 KB, recursive partitioning not needed for relations of < 1 GB with memory size of 2MB, or relations of < 36 GB with memory of 12 MB

Algorithm Selection for Other Operations



Other Operations

- **Duplicate elimination** can be implemented via hashing or sorting.
 - On sorting duplicates will come adjacent to each other, and all but one set of duplicates can be deleted.
 - *Optimization*: duplicates can be deleted during run generation as well as at intermediate merge steps in external sort-merge.
 - Hashing is similar – duplicates will come into the same bucket.
- **Projection**:
 - perform projection on each tuple
 - followed by duplicate elimination.

Select b,c,f from A
Union
Select a,q,r from B



Other Operations : Aggregation

- **Aggregation** can be implemented in a manner similar to duplicate elimination.
 - **Sorting** or **hashing** can be used to bring tuples in the same group together, and then the aggregate functions can be applied on each group.
 - Optimization: **partial aggregation**
 - combine tuples in the same group during run generation and intermediate merges, by computing partial aggregate values
 - For count, min, max, sum: keep aggregate values on tuples found so far in the group.
 - When combining partial aggregate for count, add up the partial aggregates
 - For avg, keep sum and count, and divide sum by count at the end

Evaluation



Evaluation of Expressions

- So far: we have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
 - **Materialization**: generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
 - **Pipelining**: pass on tuples to parent operations even as an operation is being executed
- We study above alternatives in more detail

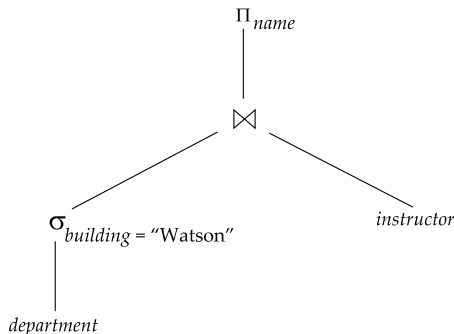


Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store

$$\sigma_{building="Watson"}(department)$$

then compute the store its join with *instructor*, and finally compute the projection on *name*.





Materialization (Cont.)

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
 - Our cost formulas for operations ignore cost of writing results to disk, so
 - Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk
- **Double buffering:** use two output buffers for each operation, when one is full write it to disk while the other is getting filled
 - Allows overlap of disk writes with computation and reduces execution time



Pipelining

- **Pipelined evaluation:** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of
$$\sigma_{building="Watson"}(department)$$
 - instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**



Pipelining (Cont.)

- In **demand driven** or **lazy** evaluation
 - system repeatedly requests next tuple from top level operation
 - Each operation requests next tuple from children operations as required, in order to output its next tuple
 - In between calls, operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining
 - Operators produce tuples eagerly and pass them up to their parents
 - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
 - if buffer is full, child waits till there is space in the buffer, and then generates more tuples
 - System schedules operations that have space in output buffer and can process more input tuples
- Alternative name: **pull** and **push** models of pipelining



Pipelining (Cont.)

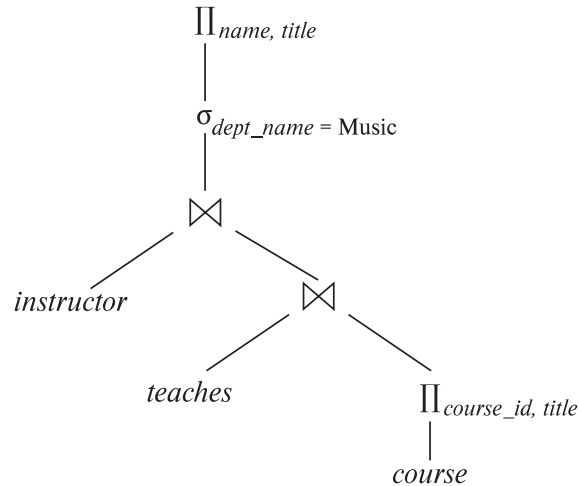
- Implementation of demand-driven pipelining
 - Each operation is implemented as an **iterator** implementing the following operations
 - **open()**
 - E.g., file scan: initialize file scan
 - state: pointer to beginning of file
 - E.g., merge join: sort relations;
 - state: pointers to beginning of sorted relations
 - **next()**
 - E.g., for file scan: Output next tuple, and advance and store file pointer
 - E.g., for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.
 - **close()**

Query Optimization (1)

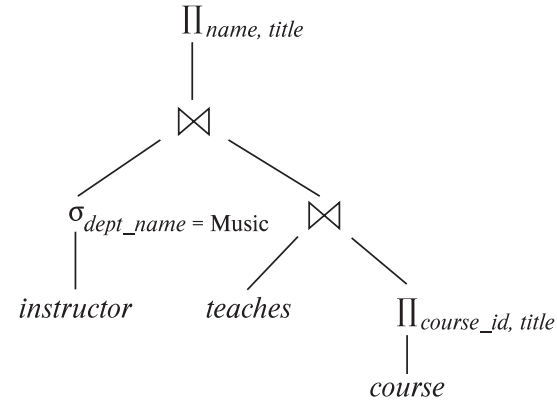


Introduction

- Alternative ways of evaluating a given query
 - Equivalent expressions
 - Different algorithms for each operation



(a) Initial expression tree

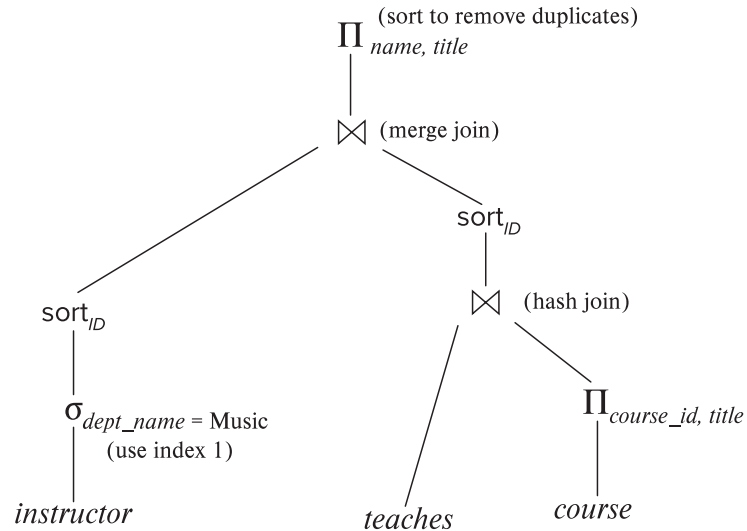


(b) Transformed expression tree



Introduction (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



- Find out how to view query execution plans on your favorite database



Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - E.g., seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 2. Annotate resultant expressions to get alternative query plans
 3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
 - Statistical information about relations. Examples:
 - number of tuples, number of distinct values for an attribute
 - Statistics estimation for intermediate results
 - to compute cost of complex expressions
 - Cost formulae for algorithms, computed using statistics



Viewing Query Evaluation Plans

- Most database support **explain** <query>
 - Displays plan chosen by query optimizer, along with cost estimates
 - Some syntax variations between databases
 - Oracle: **explain plan for** <query> followed by **select * from** table (*dbms_xplan.display*)
 - SQL Server: **set showplan_text on**
- Some databases (e.g. PostgreSQL) support **explain analyse** <query>
 - Shows actual runtime statistics found by running the query, in addition to showing the plan
- Some databases (e.g. PostgreSQL) show cost as *f..l*
 - *f* is the cost of delivering first tuple and *l* is cost of delivering all results

DFF Note: Show EXPLAIN in MySQLWorkbench



Generating Equivalent Expressions

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - Note: order of tuples is irrelevant
 - we don't care if they generate different results on databases that violate integrity constraints
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice versa



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \dots \subseteq L_n$

4. Selections can be combined with Cartesian products and theta joins.

a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$



Explanation

- $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E)) -$
 - What is going on here?
 - Why wouldn't we just go through the table with one scan?
- Consider Lahman's Baseball Database
 - `select * from people where throws="L" and nameLast="Williams";`
 - people has about 20K rows → a simple scan cost is $O(20K)$
 - Assume there is an index on throws and an index on nameLast.
 - The engine could first use the index to get matching rows producing a smaller table, and then scan the smaller result set. But which index?
 - *Most Selective Index*

```
select count(*) as no_of_rows, count(distinct nameLast) as nameLastValue,  
       count(distinct throws) as throwsValues,  
       count(*) / (count(distinct nameLast)) as nameLastSelectivity,  
       count(*) / (count(distinct throws)) as throwsSelectivity;
```

	☐ no_of_rows ▾	☐ nameLastValue ▾	☐ throwsValues ▾	☐ nameLastSelectivity ▾	☐ throwsSelectivity ▾	
1	19878	10135	3	1.9613	6626.0000	



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

- (b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

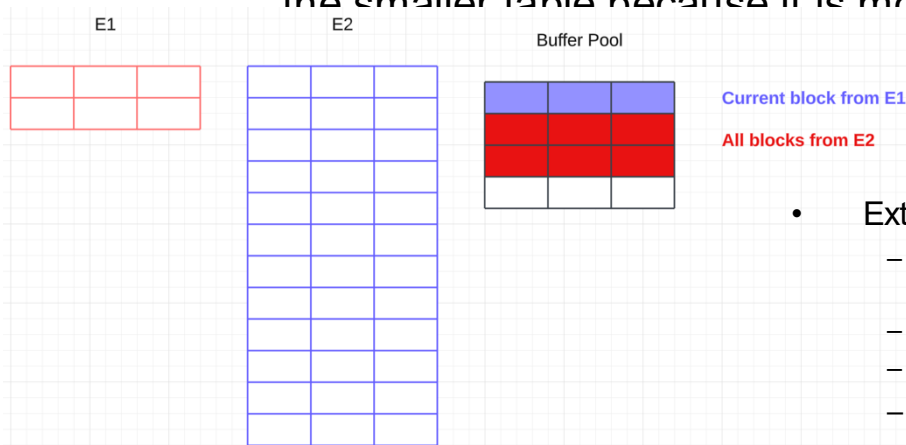
These make sense when you realize having the smaller table on the right can improve performance.

This one only makes sense when you understand another rule covered in a couple of slides.



Explanation

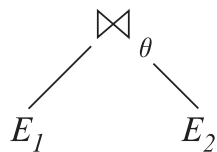
- $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$
- Assume there are no indexes → Nested loop join →
 - The engine scans the left table one time.
 - The engine scans the right table once for each row in the left table.
 - If one of the tables is (much) smaller than the other, the engine prefers scanning the smaller table because it is more efficient WRT to buffer pool usage.



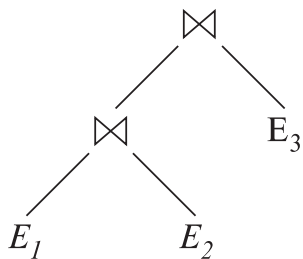
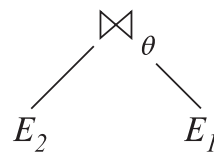
- Extreme example
 - $E_1 \bowtie E_2$ reads 2 blocks from E_1 and access 2×12 blocks from E_2 ,
 - But the engine can only have 2 or 3 in the buffer pool →
 - Approximately $2 + 24/3 = 10$ block los.
 - $E_2 \bowtie E_1$ accesses 12 blocks from E_2 and 12×2 blocks from E_1
 - But all of E_1 fits in the buffer pool → 14 block I/Os



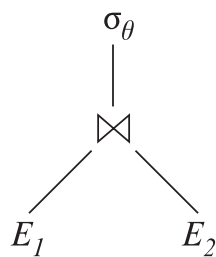
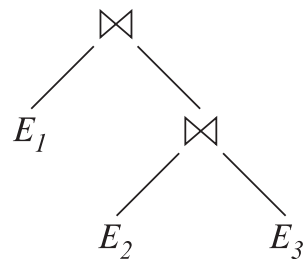
Pictorial Depiction of Equivalence Rules



Rule 5
 $\longleftrightarrow$

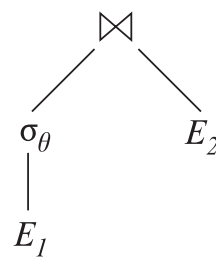


Rule 6.a
 $\longleftrightarrow$



Rule 7.a
 $\longleftrightarrow$

If θ only has
 attributes from E_1





Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

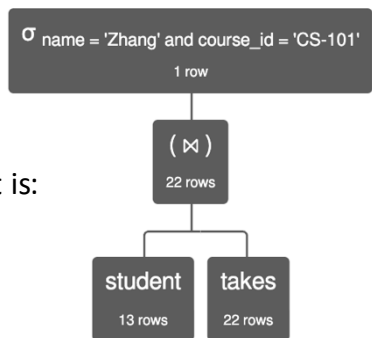


Explanation

- $\sigma_{\theta_1 \wedge \theta_2} (E_1 \bowtie E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie (\sigma_{\theta_2}(E_2))$
- $\sigma_{\text{name}='Zhang' \wedge \text{course_id}='CS-101'} (\text{student} \bowtie \text{takes})$ is equivalent to $(\sigma_{\text{name}='Zhang'} (\text{student})) \bowtie (\sigma_{\text{course_id}='CS-101'} (\text{takes}))$

Simplistically, the cost is:

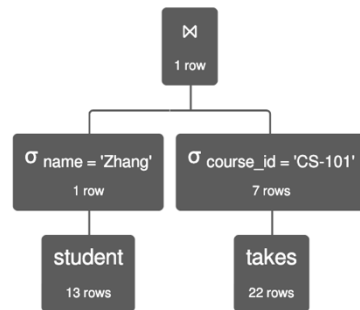
- (Join) $13 * 22 +$
- (Scan) 22



$\sigma_{\text{name}='Zhang' \text{ and } \text{course_id}='CS-101'} (\text{student} \bowtie \text{takes})$

Simplistically, the cost is:

- (Two scans) $13 + 22 +$
- (Join) $1 * 7$



$(\sigma_{\text{name}='Zhang'} (\text{student})) \bowtie (\sigma_{\text{course_id}='CS-101'} (\text{takes}))$



Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$$

(b) In general, consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$

Similar equivalences hold for outerjoin operations: $\bowtie$, $\ltimes$, and $\ltimes$

Think about this rule

- The project makes the buffer pool consumption smaller, and
- I can combine the rule with commutativity (from above).



Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$

$$E_1 \cap E_2 \equiv E_2 \cap E_1$$

(set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

a. $\sigma_{\theta}(E_1 \cup E_2) \equiv \sigma_{\theta}(E_1) \cup \sigma_{\theta}(E_2)$

b. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2)$

c. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$

d. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap E_2$

e. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - E_2$

preceding equivalence does not hold for $\cup$

I will take their word
for it.

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$



Equivalence Rules (Cont.)

13. Selection distributes over aggregation as below

$$\sigma_{\theta}(G\gamma_A(E)) \equiv G\gamma_A(\sigma_{\theta}(E))$$

provided θ only involves attributes in G

14. a. Full outerjoin is commutative:

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

- b. Left and right outerjoin are not commutative, but:

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

15. Selection distributes over left and right outerjoins as below, provided θ_1 only involves attributes of E_1

a. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv E_2 \bowtie_{\theta} (\sigma_{\theta_1}(E_1))$

16. Outerjoins can be replaced by inner joins under some conditions

a. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_1) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$

provided θ_1 is null rejecting on E_2

Oh good. More rules.





Equivalence Rules (Cont.)

Note that several equivalences that hold for joins do not hold for outerjoins

- $\sigma_{\text{year}=2017}(\text{instructor} \bowtie \text{teaches}) \neq \sigma_{\text{year}=2017}(\text{instructor} \bowtie \text{teaches})$
- Outerjoins are not associative
$$(r \bowtie s) \bowtie t \neq r \bowtie (s \bowtie t)$$
 - e.g. with $r(A,B) = \{(1,1)\}$, $s(B,C) = \{(1,1)\}$, $t(A,C) = \{\}$



Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach
 - $\Pi_{name, title}(\sigma_{dept_name = 'Music'}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$
- Transformation using rule 7a.
 - $\Pi_{name, title}((\sigma_{dept_name = 'Music'}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$
- Performing the selection as early as possible reduces the size of the relation to be joined.



Join Ordering Example

- For all relations r_1, r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity) $\bowtie$

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.



Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \\ \bowtie \Pi_{course_id, title}(course))$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{"Music"}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

$$\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches$$

first.



Enumeration of Equivalent Expressions

- Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression
- Can generate all equivalent expressions as follows:
 - Repeat
 - apply all applicable equivalence rules on every subexpression of every equivalent expression found so far
 - add newly generated expressions to the set of equivalent expressions

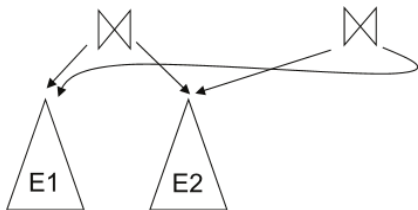
Until no new equivalent expressions are generated above

- The above approach is very expensive in space and time
 - Two approaches
 - Optimized plan generation based on transformation rules
 - Special case approach for queries with only selections, projections and joins

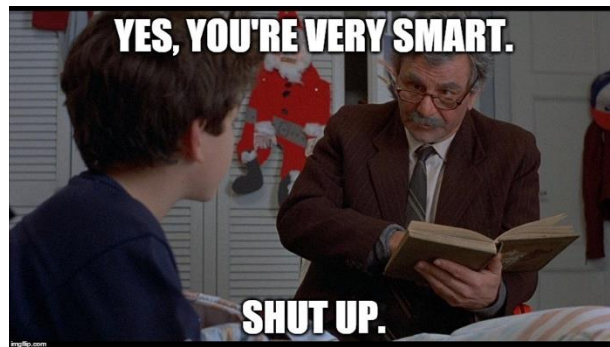


Implementing Transformation Based Optimization

- Space requirements reduced by sharing common sub-expressions:
 - when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers
 - E.g., when applying join commutativity



- Same sub-expression may get generated multiple times
 - Detect duplicate sub-expressions and share one copy
- Time requirements are reduced by not generating all expressions
 - Dynamic programming
 - We will study only the special case of dynamic programming for join order optimization





Cost Estimation

- Cost of each operator computer as described in Chapter 15
 - Need statistics of input relations
 - E.g., number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
 - Need to estimate statistics of expression results
 - To do so, we require additional statistics
 - E.g., number of distinct values for an attribute
- More on cost estimation later

Remember algorithm selection applies to the operators in each of the possible trees.



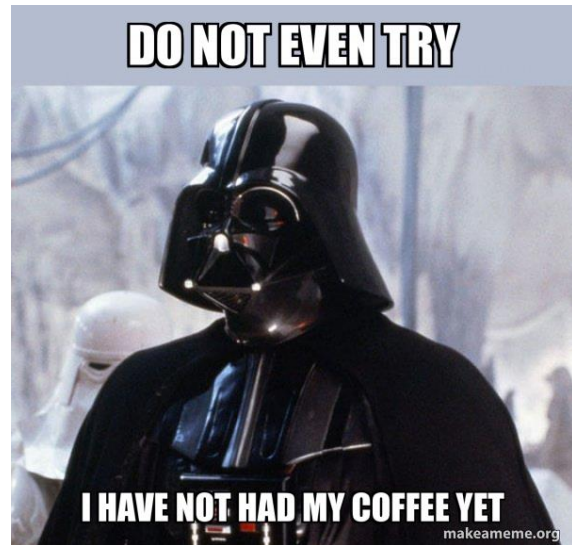
Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
 - merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
 - nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.



Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S using selections on S and indices (if any) on S else for each
    non-empty subset S1 of S such that S1  $\neq S$ 
      P1 = findbestplan(S1)
      P2 = findbestplan(S - S1)
      for each algorithm A for joining results of P1 and P2
        ... compute plan and cost of using A (see next page) ..
        if cost < bestplan[S].cost
          bestplan[S].cost = cost
          bestplan[S].plan = plan;
  return bestplan[S]
```



NoSQL

Reminder

Simplistic Classification (<https://medium.com/swlh/4-types-of-nosql-databases-d88ad21f7d3b>)

Relational is the foundational model.

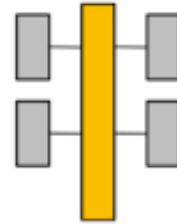
We covered graphs and examples.

SQL Database

Relational



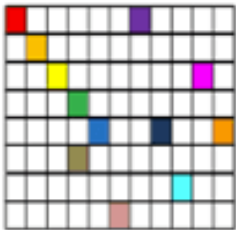
Analytical (OLAP)



We will see OLAP in a future lecture.

NoSQL Database

Column-Family



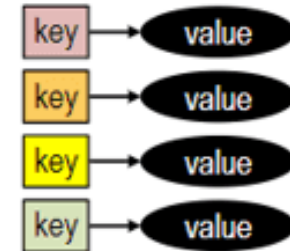
Graph



Document



Key-Value



Subject of this lecture and part of HW3

MongoDB Again

MongoDB Concepts

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key _id provided by MongoDB itself)
Database Server, Client, Tools, Packages	
mysql/Oracle	mongod
mysql/sqlplus	mongo
DataGrip	Compass
pymysql	pymongo

Core Operations

Basic Operations:

- Create database
- Create collection
- Create-Retrieve-Update-Delete (CRUD):
 - Create: insert()
 - Retrieve:
 - find()
 - find_one()
 - Update: update()
 - Delete: remove()

More Advanced Concepts:

- Limit
- Sort
- Aggregation Pipelines
 - Merge
 - Union
 - Lookup
 - Match
 - Merge
 - Sample
 -

We will just cover the basics for now and may cover more things in HW or other lectures.

find()

- Note:
 - MongoDB uses a more pymysql approach, e.g. an API, than pure declarative languages like SQL.
 - The parameters for find() are where the declarative language appears.
- The basic forms of find() and find_one() have two parameters:
 - *filter expression*
 - *Project expression*
- You can use the Compass tool and screen captures for some HW and exam answers.
- What if I want the answer in a Jupyter Notebook?

The screenshot shows the MongoDB Compass interface for the 'GOT.seasons' collection. The 'Documents' tab is selected, and the filter bar contains the expression: `{ "episodes.scenes.location": "The Dothraki Sea" }`. The project bar contains the expression: `{ season: 1, "episodes.episodeNum": 1, "episodes.episodeLink": 1, "episodes.episodeTitle": 1 }`. The sort bar contains the expression: `{ field: -1 }`. The collation bar contains the expression: `{ locale: 'simple' }`. The interface also shows document statistics: 8 documents, 1.0MB total size, 130.2KB avg. size, 1 index, 20.0KB total size, 20.0KB avg. size. The first document is expanded, showing a nested structure for season 1 with episodes.

Generate Code

Export Query To Language

My Query:

```
1 {
2   "episodes.scenes.location": "The Dothraki Sea"
3 }
```

Export Query To: PYTHON 3

```
1 # Requires the PyMongo package.
2 # https://api.mongodb.com/python/current
3
4 client = MongoClient('mongodb://localhost:27017/?
5 filter={
6   'episodes.scenes.location': 'The Dothraki Sea
7 }
8 project={
9   'season': 1,
10  'episodes.episodeNum': 1,
11  'episodes.episodeLink': 1,
12  'episodes.episodeTitle': 1
13 }
14
15 result = client['GOT']['seasons'].find(
16   filter=filter,
```

☐ Include Import Statements

☒ Include Driver Syntax

CLOSE

- Choose Export to Language.
- Copy into the notebook.
- The export has an option to include all the connection setup, choosing DB,
- Switch to Notebook

Result is not Quite You Expect

GOT.seasons

DOCUMENTS 8 TOTAL SIZE 1.0MB AVG. SIZE 130.2KB INDEXES 1 TOTAL SIZE 20.0KB AVG. SIZE 20.0KB

Documents Aggregations Schema Explain Plan Indexes Validation

FILTER `{ "episodes.scenes.location": "The Dothraki Sea" }` **OPTIONS** **FIND** **RESET** **...**

PROJECT `{ season: 1, "episodes.episodeNum": 1, "episodes.episodeLink": 1, "episodes.episodeTitle": 1 }`

SORT `{ field: -1 }` **MAX TIME MS** 60000

COLLATION `{ locale: 'simple' }` **SKIP** 0 **LIMIT** 0

VIEW **JSON** **TABLE** **...**

Displaying documents 1 - 3 of 3 **REFRESH**

```
{
  "_id": ObjectId("60577e50c68b67110968b6d1"),
  "season": "1",
  "episodes": Array
    ▾ 0: Object
      episodeNum: 1
      episodeTitle: "Winter Is Coming"
      episodeLink: "/title/tt1480055/"
    ▸ 1: Object
    ▸ 2: Object
    ▸ 3: Object
      episodeNum: 4
      episodeTitle: "Cripples, Bastards, and Broken Things"
      episodeLink: "/title/tt1829963/"
    ▸ 4: Object
    ▸ 5: Object
    ▸ 6: Object
    ▸ 7: Object
    ▸ 8: Object
    ▸ 9: Object
}
```

```
{
  "_id": ObjectId("60577e50c68b67110968b6d5"),
  "season": "5",
  "episodes": Array
}
```

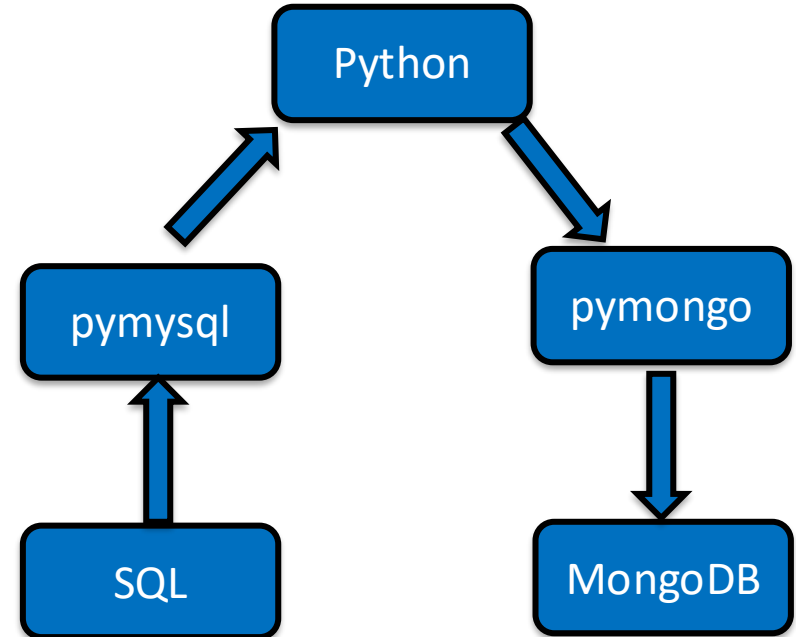
```
{
  "_id": ObjectId("60577e50c68b67110968b6d6"),
  "season": "6",
  "episodes": Array
}
```

- The query returns documents that match.
 - The document is “Large” and has and episodes and seasons.
 - If you do a \$project requesting episodes/episode content,
 - You get all episodes in the documents that match.
 - Not just the episodes with the scene/location.
 - Projecting array elements from arrays whose elements are arrays is complex and baffling.
- You also get back something (a cursor) that is iterable.

More Fun – Data Types

MongoDB supports many datatypes. Some of them are –

- **String** – This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.
- **Integer** – This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.
- **Boolean** – This type is used to store a boolean (true/ false) value.
- **Double** – This type is used to store floating point values.
- **Min/ Max keys** – This type is used to compare a value against the lowest and highest BSON elements.
- **Arrays** – This type is used to store arrays or list or multiple values into one key.
- **Timestamp** – timestamp. This can be handy for recording when a document has been modified or added.
- **Object** – This datatype is used for embedded documents.
- **Null** – This type is used to store a Null value.
- **Symbol** – This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
- **Date** – This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.
- **Object ID** – This datatype is used to store the document's ID.
- **Binary data** – This datatype is used to store binary data.
- **Code** – This datatype is used to store JavaScript code into the document.
- **Regular expression** – This datatype is used to store regular expression.



(Some) MongoDB CRUD Operations

- Create:
 - `db.collection.insertOne()`
 - `db.collection.insertMany()`
- Retrieve:
 - `db.collection.find()`
 - `db.collection.findOne()`
 - `db.collection.findOneAndUpdate()`
 -
- Update:
 - `db.collection.updateOne()`
 - `db.collection.updateMany()`
 - `db.collection.replaceOne()`
- Delete:
 - `db.collection.deleteOne()`
 - `db.collection.deleteMany()`

pymongo maps the camel case to `_`, e.g.

- `findOne()`
- `find_one()`

There are good online tutorials:

- https://www.tutorialspoint.com/python_data_access
- <https://www.tutorialspoint.com/mongodb/index.htm>

(Some) MongoDB Pipeline Operators

<https://www.slideshare.net/mongodb/s01-e04-analytics>

Aggregation operators

- Pipeline and Expression operators

Pipeline	Expression	Arithmetic	Conditional
\$match	\$addToSet	\$add	\$cond
\$sort	\$first	\$divide	\$ifNull
\$limit	\$last	\$mod	
\$skip	\$max	\$multiply	
\$project	\$min	\$subtract	
\$unwind	\$avg		Variables
\$group	\$push		\$let
\$geoNear	\$sum		\$map
\$text			
\$search			

Tip: Other operators for date, time, boolean and string manipulation

MongoDB Checkpoint

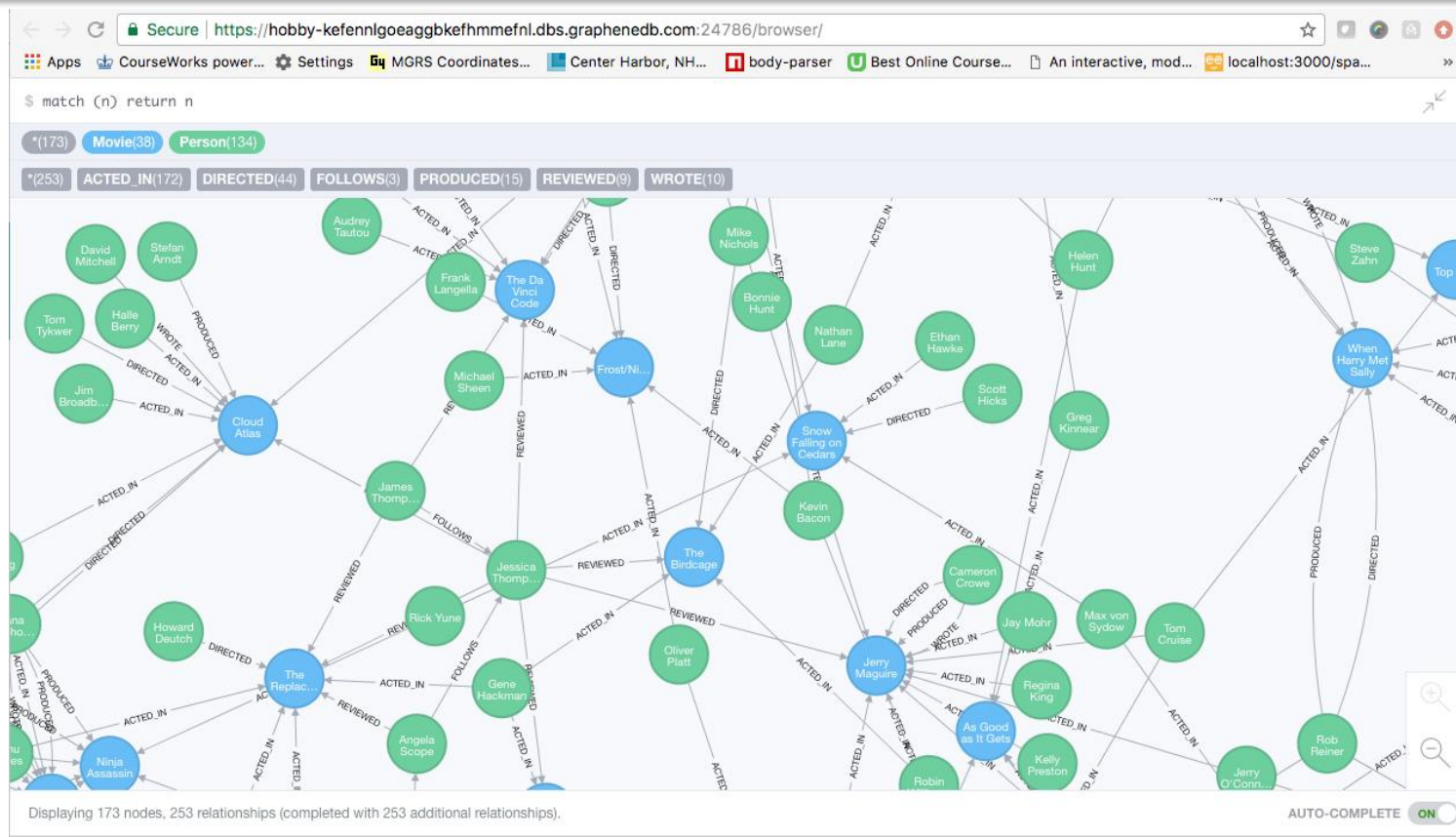
- You can see that MongoDB has powerful, sophisticated
 - Operators
 - Expressions
 - Pipelines
- We have
 - Indexes
 - Replicas
 - Embedded documents
 -
- We will explore a little more in subsequent lectures, homework,
- ~~You will have to install MongoDB and Compass for HW4 and final exam.~~

- <https://www.mongodb.com/docs/manual/aggregation/>
- <https://www.mongodb.com/docs/manual/tutorial/aggregation-complete-examples/>
- Let's look at some examples.

Let's Do Some Examples

Neo4j

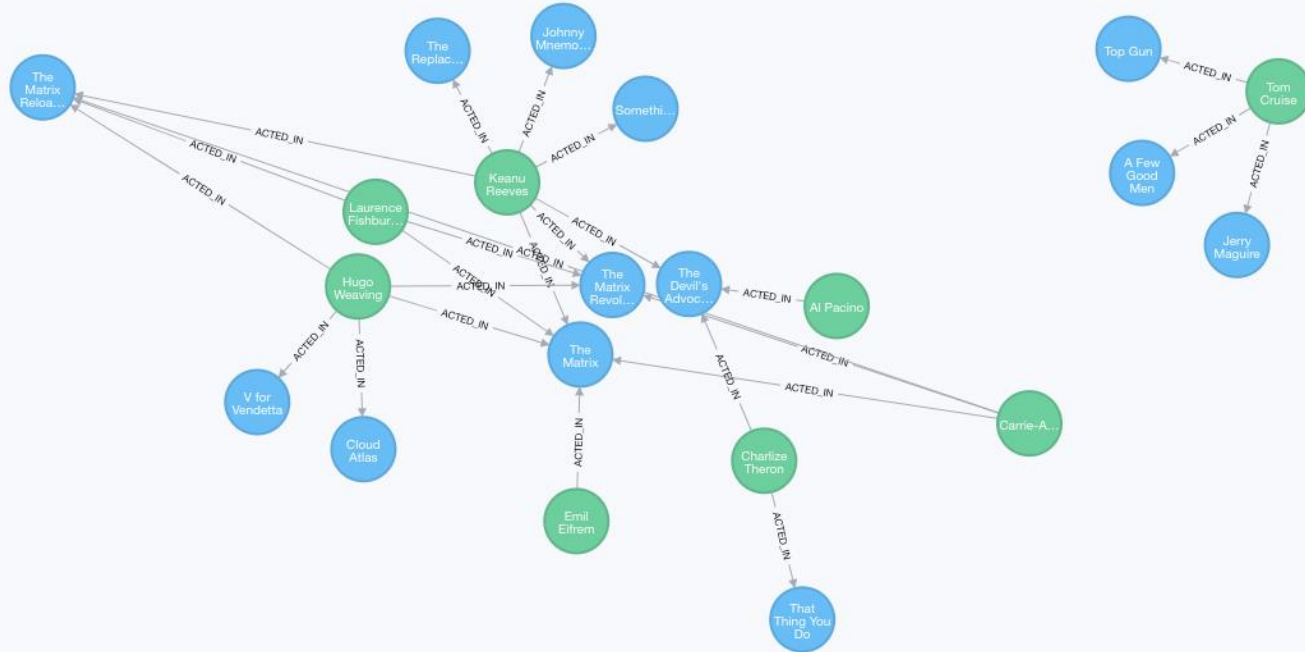
A Graph Database (Sample)



```
$ MATCH p=()-[r:ACTED_IN]->() RETURN p LIMIT 25
```

*** (21)** **Movie (13)** **Person (8)**

*{25} ACTED_IN(25)



Displaying 21 nodes, 25 relationships.

AUTO-COMPLETE ☒

Big Deal. That is just a JOIN.

- Yup. But that is simple.
- Try writing the queries below in SQL.

The Movie Graph

Recommend

Let's recommend new co-actors for Tom Hanks. A basic recommendation approach is to find connections past an immediate neighborhood which are themselves well connected.

For Tom Hanks, that means:

1. Find actors that Tom Hanks hasn't yet worked with, but his co-actors have.
2. Find someone who can introduce Tom to his potential co-actor.

Extend Tom Hanks co-actors, to find co-co-actors who haven't work with Tom Hanks...

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
WHERE NOT (tom)-[:ACTED_IN]->(m2)
RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```

Find someone to introduce Tom Hanks to Tom Cruise

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
RETURN tom, m, coActors, m2, cruise
```

Recommend

```
1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
2     (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
3 WHERE NOT (tom)-[:ACTED_IN]->(m2)
4 RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```



\$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors) ...



	Recommended	Strength
Rows	Tom Cruise	5
A	Zach Grenier	5
Text	Helen Hunt	4
</>	Cuba Gooding Jr.	4
Code	Keanu Reeves	4
	Tom Skerritt	3
	Carrie-Anne Moss	3
	Val Kilmer	3
	Bruno Kirby	3
	Philip Seymour Hoffman	3
	Billy Crystal	3
	Carrie Fisher	3


```

1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors),
2     (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
3 RETURN tom, m, coActors, m2, cruise

```

\$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})

Graph

*(13) Movie(8) Person(5)

*(16) ACTED_IN(16)

Rows

Text

Code



Which actors have worked with both Tom Hanks and Tom Cruise?

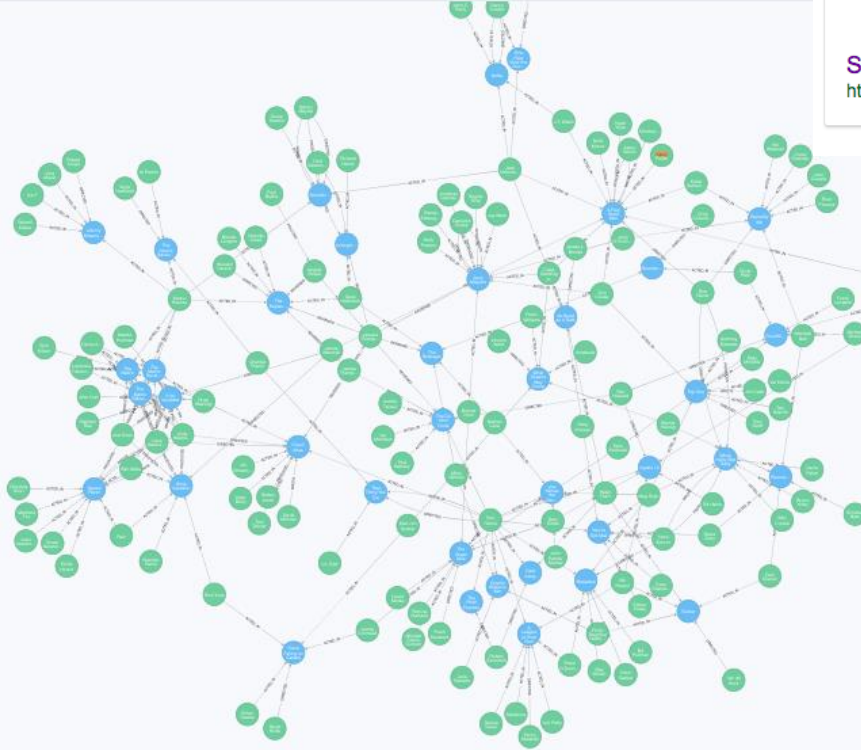
Displaying 13 nodes, 16 relationships (completed with 16 additional relationships).

AUTO-COMPLETE ☒

```
$ MATCH (s:Person { name: 'Kevin Bacon' })-[*0..6]-(m) return s,m
```

*(171) Movie(38) Person(133)

*(253) ACTED_IN(172) DIRECTED(44) FOLLOWS(3) PRODUCED(15) REVIEWED(9) WROTE(10)



Six Degrees of Kevin Bacon is a parlour game based on the "six degrees of separation" concept, which posits that any two people on Earth are six or fewer acquaintance links apart. Movie buffs challenge each other to find the shortest path between an arbitrary actor and prolific actor **Kevin Bacon**.



[Six Degrees of Kevin Bacon - Wikipedia](https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon

About this result

Feedback

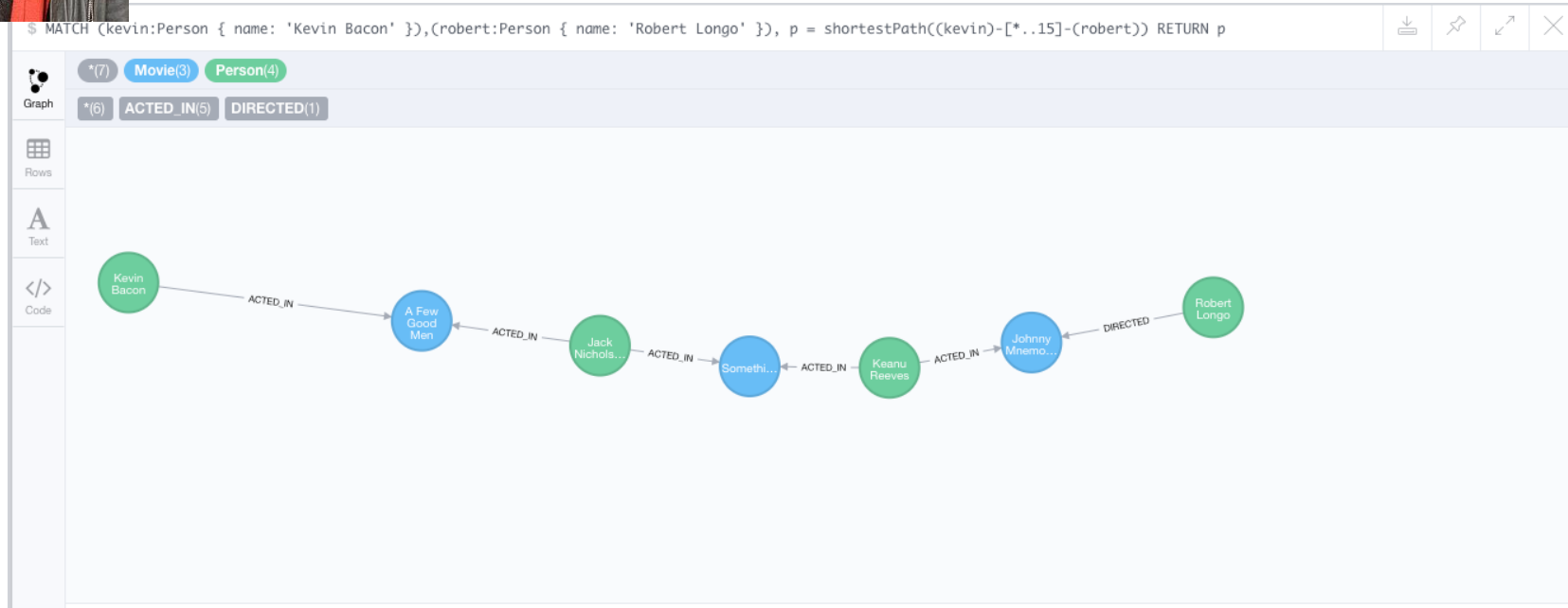
Six Degrees of Kevin Bacon

Game





How do you get from Kevin Bacon to Robert Longo?



Some Complex Examples

Web Application and REST

Full Stack Application

Full Stack Developer Meaning & Definition

In technology development, full stack refers to an entire computer system or application from the **front end** to the **back end** and the **code** that connects the two. The back end of a computer system encompasses "behind-the-scenes" technologies such as the **database** and **operating system**. The front end is the **user interface** (UI). This end-to-end system requires many ancillary technologies such as the **network**, **hardware**, **load balancers**, and **firewalls**.

FULL STACK WEB DEVELOPERS

Full stack is most commonly used when referring to **web developers**. A full stack web developer works with both the front and back end of a website or application. They are proficient in both front-end and back-end **languages** and frameworks, as well as server, network, and **hosting** environments.

Full-stack developers need to be proficient in languages used for front-end development such as **HTML**, **CSS**, **JavaScript**, and third-party libraries and extensions for Web development such as **JQuery**, **SASS**, and **REACT**. Mastery of these front-end programming languages will need to be combined with knowledge of UI design as well as customer experience design for creating optimal front-facing websites and applications.

<https://www.webopedia.com/definitions/full-stack/>

Full Stack Web Developer

A full stack web developer is a person who can develop both **client** and **server** software.

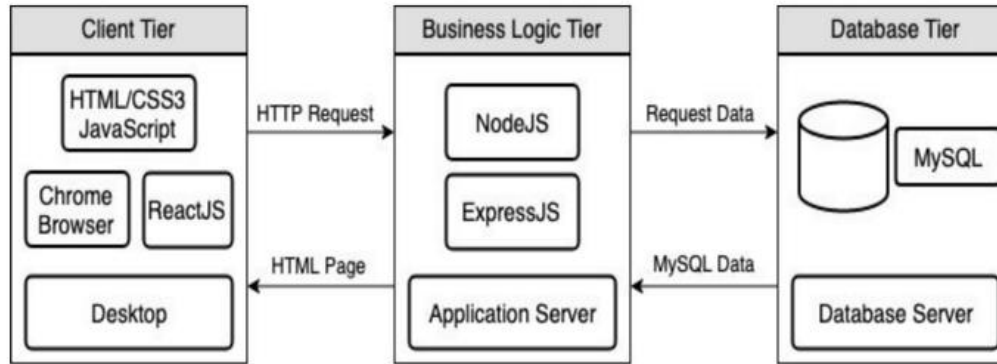
In addition to mastering HTML and CSS, he/she also knows how to:

- Program a **browser** (like using JavaScript, jQuery, Angular, or Vue)
- Program a **server** (like using PHP, ASP, Python, or Node)
- Program a **database** (like using SQL, SQLite, or MongoDB)

https://www.w3schools.com/whatis/whatis_fullstack.asp

- There are courses that cover topics:
 - COMS W4153: Advanced Software Engineering
 - COMS W4111: Introduction to Databases
 - COMS W4170 - User Interface Design
- This course will focus on cloud realization, microservices and application patterns,
- Also, I am not great at UIs We will not emphasize or require a lot of UI work.

Full Stack Web Application



M = Mongo
E = Express
R = React
N = Node

I start with FastAPI and MySQL,
but all the concepts are the same.

<https://levelup.gitconnected.com/a-complete-guide-build-a-scalable-3-tier-architecture-with-mern-stack-es6-ca129d7df805>

- My preferences are to replace React with Angular, and Node with Flask.
- There are three projects to design, develop, test, deploy,
 1. Browser UI application.
 2. Microservice.
 3. Database.
- We will initial have two deployments: local machine, virtual machine.
We will ignore the database for step 1.

Some Terms

- A web application server or web application framework: “A web framework (WF) or web application framework (WAF) is a software framework that is designed to support the development of web applications including web services, web resources, and web APIs. Web frameworks provide a standard way to build and deploy web applications on the World Wide Web. Web frameworks aim to automate the overhead associated with common activities performed in web development. For example, many web frameworks provide libraries for database access, templating frameworks, and session management, and they often promote code reuse.”
(https://en.wikipedia.org/wiki/Web_framework)
- REST: “REST (Representational State Transfer) is a software architectural style that was created to guide the design and development of the architecture for the World Wide Web. REST defines a set of constraints for how the architecture of a distributed, Internet-scale hypermedia system, such as the Web, should behave. The REST architectural style emphasises uniform interfaces, independent deployment of components, the scalability of interactions between them, and creating a layered architecture to promote caching to reduce user-perceived latency, enforce security, and encapsulate legacy systems.[1]

REST has been employed throughout the software industry to create stateless, reliable web-based applications.”
(<https://en.wikipedia.org/wiki/REST>)

Some Terms

- OpenAPI: “The OpenAPI Specification, previously known as the Swagger Specification, is a specification for a machine-readable interface definition language for describing, producing, consuming and visualizing web services.” (https://en.wikipedia.org/wiki/OpenAPI_Specification)
- Model: “A model represents an entity of our application domain with an associated type.” (<https://medium.com/@nicola88/your-first-openapi-document-part-ii-data-model-52ee1d6503e0>)
- Routers: “What fastapi docs says about routers: If you are building an application or a web API, it’s rarely the case that you can put everything on a single file. FastAPI provides a convenience tool to structure your application while keeping all the flexibility.” (<https://medium.com/@rushikeshnaik779/routers-in-fastapi-tutorial-2-adf3e505fdca>)
- Summary:
 - These are general concepts, and we will go into more detail in the semester.
 - FastAPI is a specific technology for Python.
 - There are many other frameworks applicable to Python, NodeJS/TypeScript, Go, C#, Java,
 - They all surface similar concepts with slightly different names.

REST (<https://restfulapi.net/>)

What is REST

- REST is acronym for **RE**presentational **State** Transfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous [dissertation](#).
- Like any other architectural style, REST also does have it's own [6 guiding constraints](#) which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client-server** – By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot “see” beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

Resources

Resources are an abstraction. The application maps to create things and actions.

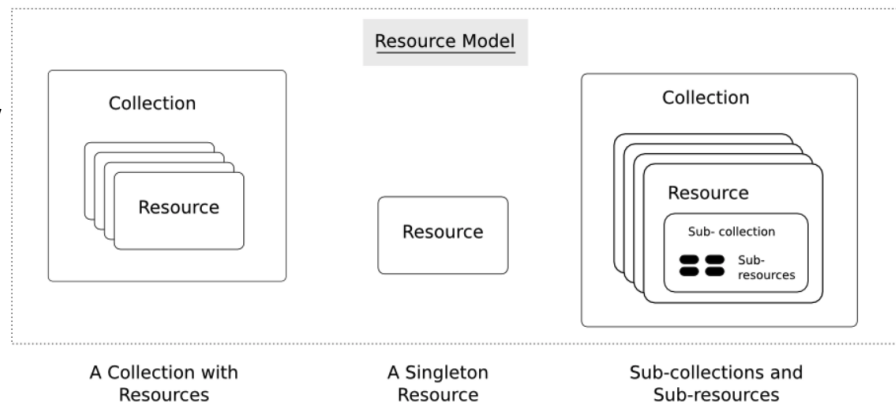
“A resource-oriented API is generally modeled as a resource hierarchy, where each node is either a *simple resource* or a *collection resource*. For convenience, they are often called a resource and a collection, respectively.

- A collection contains a list of resources of **the same type**. For example, a user has a collection of contacts.
- A resource has some state and zero or more sub-resources. Each sub-resource can be either a simple resource or a collection resource.

For example, Gmail API has a collection of users, each user has a collection of messages, a collection of threads, a collection of labels, a profile resource, and several setting resources.

While there is some conceptual alignment between storage systems and REST APIs, a service with a resource-oriented API is not necessarily a database, and has enormous flexibility in how it interprets resources and methods. For example, creating a calendar event (resource) may create additional events for attendees, send email invitations to attendees, reserve conference rooms, and update video conference schedules. (Emphasis added)

(<https://cloud.google.com/apis/design/resources#resources>)



<https://restful-api-design.readthedocs.io/en/latest/resources.html>

REST – Resource Oriented

- When writing applications, we are used to writing functions or methods:
 - `openAccount(last_name, first_name, tax_payer_id)`
 - `account.deposit(deposit_amount)`
 - `account.close()`

We can create and implement whatever functions we need.

- REST only allows four methods:

- POST: Create a resource
- GET: Retrieve a resource
- PUT: Update a resource
- DELETE: Delete a resource

That's it. That's all you get.

“The key characteristic of a resource-oriented API is that it emphasizes resources (data model) over the methods performed on the resources (functionality). A typical resource-oriented API exposes a large number of resources with a small number of methods.”
(<https://cloud.google.com/apis/design/resources>)

- A REST client needs no prior knowledge about how to interact with any particular application or server beyond a generic understanding of hypermedia.

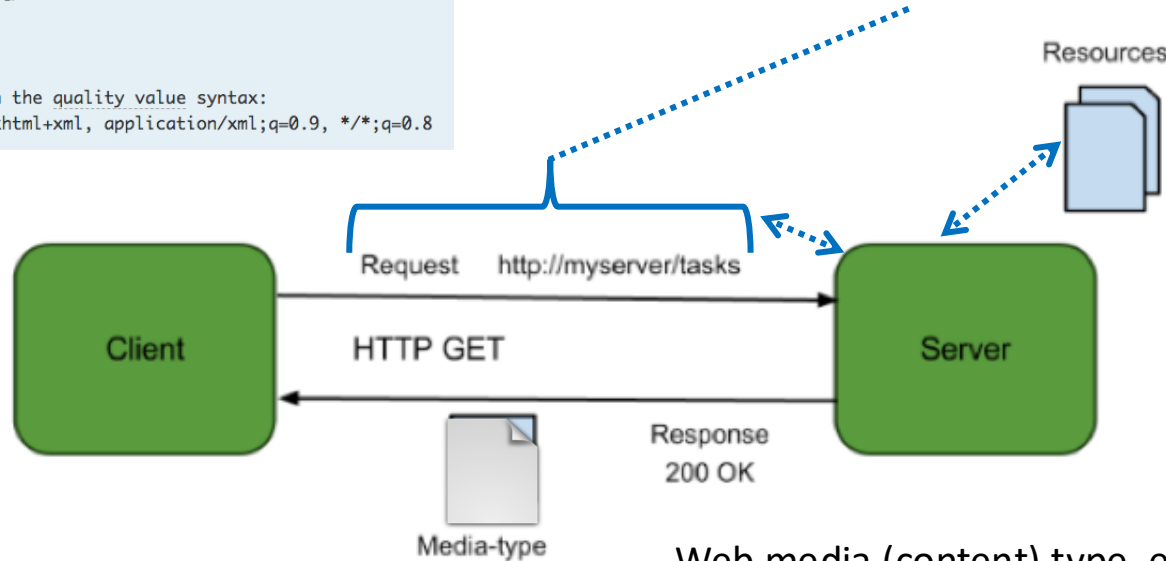
Resources, URLs, Content Types

Accept type in headers.

```
Accept: <MIME_type>/<MIME_subtype>
Accept: <MIME_type>/*
Accept: */*
```

```
// Multiple types, weighted with the quality value syntax:
Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
```

- Relative URL identifies “resource” on the server.
- Server implementation maps abstract resource to tangible “thing,” file, DB row, ... and any application logic.

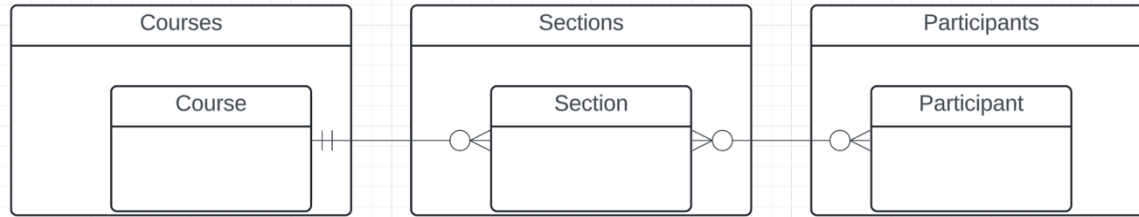


Client may be
Browser
Mobile device
Other REST Service
... ..

Web media (content) type, e.g.

- text/html
- application/json

Resources and APIs



- Base resources, paths and methods:
 - /courses: GET, ~~POST~~
 - /courses/<id>: GET, PUT, ~~DELETE~~
 - /sections: GET, ~~POST~~
 - /sections/<id>: GET, PUT, ~~DELETE~~
 - /participants: GET, ~~POST~~
 - /participants/<id>: GET, PUT, ~~DELETE~~
- There are relative, navigation paths:
 - /courses/<id>/sections
 - /participants/<id>/sections
 - etc.
- GET on resources that are collections may also have query parameters.
- There are two approaches to defining API
 - Start with OpenAPI document and produce an implementation template.
 - Start with annotated code and generate API document.
- In either approach, I start with *models*.
- Also,
 - I lack the security permission to update CourseWorks.
 - I can choose to not surface the methods or raise and exception.

Data Modeling Concepts and REST

Almost any data model has the same core concepts:

- Types and instances:
 - Entity Type: A definition of a type of thing with properties and relationships.
 - Entity Instance: A specific instantiation of the Entity Type
 - Entity Set Instance: An Entity Type that:
 - Has properties and relationships like any entity, but ...
 - Has at least one *special relationship* – **contains**.
- Operations, minimally CRUD, that manipulate entity types and instances:
 - Create
 - Retrieve
 - Update
 - Delete
 - Reference/Identify/... ..
 - Host/database/table/pk

What is REST architecture?

REST stands for REpresentational State Transfer. REST is web standards based architecture and uses HTTP Protocol. It revolves around resource where every component is a resource and a resource is accessed by a common interface using HTTP standard methods. REST was first introduced by Roy Fielding in 2000.

In REST architecture, a REST Server simply provides access to resources and REST client accesses and modifies the resources. Here each resource is identified by URIs/ global IDs. REST uses various representation to represent a resource like text, JSON, XML. JSON is the most popular one.

HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

- ▣ **GET** – Provides a read only access to a resource.
- ▣ **POST** – Used to create a new resource.
- ▣ **DELETE** – Used to remove a resource.
- ▣ **PUT** – Used to update a existing resource or create a new resource.

Introduction to RESTful web services

A web service is a collection of open protocols and standards used for exchanging data between applications or systems. Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer. This interoperability (e.g., between Java and Python, or Windows and Linux applications) is due to the use of open standards.

Web services based on REST Architecture are known as RESTful web services. These webservices uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, provides resource representation such as JSON and set of HTTP Methods.

Creating RESTful Webservice

In next chapters, we'll create a webservice say user management with following functionalities –

Sr.No.	URI	HTTP Method	POST body	Result
1	/UserService/users	GET	empty	Show list of all the users.
2	/UserService/addUser	POST	JSON String	Add details of new user.
3	/UserService/getUser/:id	GET	empty	Show details of a user.